NLP Group Assignment 2

Question 1: Attention-Based Neural Machine Translation

Main Objective

Build an English-to-Hindi Neural Machine Translation (NMT) system using an LSTM Encoder–Decoder. First implement a baseline without Attention, then extend it with an Attention mechanism. Train and compare both systems and visualize attention weights to investigate which parts of the English sentence the model focuses on while generating Hindi.

> **Important:** The goal is not only to obtain translations, but also to investigate how Attention changes NMT performance and model behavior.

Corpus to Use

Use the IIT Bombay English–Hindi Parallel Corpus. Use the training data for training, development data for tuning/model selection, and the test set for final evaluation. A manageable training subset (e.g., 20,000–50,000 sentence pairs) may be used depending on computational resources.

Official corpus: `https://www.cfilt.iitb.ac.in/iitb_parallel/`

Step-by-Step Breakdown

Part 1 — Data Preparation

- Load the English–Hindi parallel sentences.
- Remove empty or malformed sentence pairs and normalize the text.
- Tokenize the English and Hindi sentences.
- Add <SOS> and <EOS> special tokens.
- Build source and target vocabularies.
- Convert tokens into integer IDs and pad sequences.

Report: number of training pairs used, English vocabulary size, Hindi vocabulary size, maximum English sentence length, and maximum Hindi sentence length.

```
Example
English: I am going to school.
Hindi:   मैं स्कूल जा रहा हूँ।

English: <SOS> I am going to school . <EOS>
Hindi:   <SOS> मैं स्कूल जा रहा हूँ । <EOS>
```

Part 2 — Baseline NMT Model

Build a standard Encoder–Decoder NMT model without Attention.

```
English Sentence → Embedding → Encoder LSTM → Final Hidden State → Decoder LSTM →
Dense + Softmax → Hindi Sentence
```

Train using teacher forcing. Record training loss, validation loss, training accuracy, and validation accuracy. This model is the baseline.

Part 3 — Identify the Limitation of the Baseline

Investigate what happens when the input English sentence becomes longer. The baseline compresses the source sentence into a fixed representation before decoding.

- Why can this become difficult for long sentences?
- Does the baseline perform differently on short and long sentences?
- Are important words from the beginning sometimes lost or poorly represented?

Divide test sentences into at least short and long groups and report performance separately. This motivates Attention.

Part 4 — Add Attention Mechanism

Modify the baseline by introducing Attention. Instead of using only the final encoder state, retain all encoder hidden states h_1, h_2, …, h_n.

$$e_{t,i} = \text{score}(s_{t-1}, h_i)$$
$$\alpha_{t,i} = \exp(e_{t,i}) \;/\; \Sigma_j \exp(e_{t,j})$$
$$c_t = \Sigma_i \alpha_{t,i} h_i$$

The decoder uses the context vector while generating the next Hindi token.

$$\text{English} \rightarrow \text{Embedding} \rightarrow \text{Encoder LSTM} \rightarrow h_1, h_2, \ldots, h_n \rightarrow \text{Attention} \rightarrow \text{Context Vector} \rightarrow \text{Decoder LSTM} \rightarrow \text{Dense} + \text{Softmax} \rightarrow \text{Hindi}$$

> **Important:** Do not simply use a pre-built high-level translation model. Implement the Attention mechanism so that its weights can be inspected.

Part 5 — Train and Compare the Two Models

Model	Architecture
Model A	LSTM Encoder–Decoder WITHOUT Attention
Model B	LSTM Encoder–Decoder WITH Attention

Keep major training conditions comparable. Plot training loss, validation loss, training accuracy, and validation accuracy. Report the effect of adding Attention.

Part 6 — Visualize What the Model is Attending To

For selected translations, extract attention weights $\alpha_{t,i}$ and create a heatmap. The x-axis should contain English tokens and the y-axis should contain generated Hindi tokens.

	I	am	going	home
मैं				
घर				
जाता				
हूँ				

- Which English words receive high attention?
- Does the model attend to the relevant source word when generating a Hindi word?
- Does attention move sequentially through the sentence?
- Are there cases where the model attends to multiple source words?
- Are there cases where attention appears incorrect?

Part 7 — Attention-Based Error Analysis

Select at least 5 test sentences where the attention model produces an incorrect or partially incorrect translation. For each, report the English input, reference Hindi translation, baseline prediction, attention prediction, and attention heatmap.

Classify errors where possible: missing word, extra word, incorrect word, incorrect word order, long-distance dependency, unknown/OOV word, or incorrect attention focus.

Discuss whether the attention visualization provides a useful explanation for the error.

Part 8 — Performance Analysis by Sentence Length

Divide the test set into categories such as:

- 1–5 tokens
- 6–10 tokens
- 11–15 tokens
- 16+ tokens

Calculate the translation performance of the baseline and Attention NMT for each category. Create a graph showing performance as a function of sentence length and discuss whether the difference changes with sentence length.

Part 9 — Final Comparative Analysis

- How did the Attention model compare with the baseline?
- Did Attention help more on short or long sentences?
- Which translation errors were reduced?
- What do the attention heatmaps reveal?
- Are high attention weights always associated with correct translations?
- Can attention visualization be used as an explanation of model behavior? Discuss based on your observations.
- What are the limitations of your implementation?

What to Submit

1. Data preprocessing code
2. Baseline Encoder–Decoder NMT code
3. Attention-based NMT code
4. Training and validation plots
5. Translation results
6. Attention heatmaps
7. Sentence-length analysis
8. Error analysis
9. Comparison of baseline vs. Attention model
10. Short technical report

Marking Scheme — 100 Marks

Component	Marks
Data preparation & preprocessing	10
Baseline Encoder–Decoder NMT	15
Baseline analysis	10
Attention implementation	20
Training & model comparison	15
Attention visualization	15
Error & sentence-length analysis	10
Final comparative report	5
Total	**100**

> **Core Question of the Project:** Does Attention actually improve Neural Machine Translation, and can we understand the model's translation behavior by visualizing where it attends?

Question 2: Fine-Tuning RoBERTa for Multi-Class Product Review Rating Prediction

Objective

The objective of this task is to develop and evaluate a high-performance rating prediction model by fine-tuning a pre-trained RoBERTa (Robustly Optimized BERT Pretraining Approach) model. The goal is to perform 5-class rating classification on a large-scale product review dataset, compare its performance against a strong recurrent baseline with an attention mechanism, and study how well each model respects the ordinal nature of star ratings.

Dataset

For this task, use the **Amazon Reviews (English) Star Rating Dataset**. This is a large-scale dataset for 5-class rating classification, where 1 star is "very negative" and 5 stars is "very positive". Each sample contains a review title and a review body, which must be concatenated into a single input text.

- **Dataset Source:** Can be loaded using the Hugging Face datasets library:

```
datasets.load_dataset('SetFit/amazon_reviews_multi_en')
```

- **Data Splits:** Training Set: 200,000 reviews; Validation Set: 5,000 reviews; Test Set: 5,000 reviews. The validation set must be used for model selection and early stopping; the test set must be touched only once for the final evaluation.

Problem Statement

Implement two rating classification models for the 5-class Amazon dataset:

1. **Baseline RNN Model (without Transformers):** Implement a strong baseline using a Bidirectional GRU with an additive attention pooling layer. This model should use pre-trained fastText (or GloVe) word embeddings as its input layer.
2. **RoBERTa-based Classification Model:** Implement a classification model by fine-tuning a pre-trained RoBERTa model (e.g., `roberta-base`). Instead of using only the first-token (<s>) embedding, use mean pooling over all non-padded token embeddings of the last hidden layer, followed by a classification head, and fine-tune the entire model end-to-end.

Compare the performance of both models using Accuracy, Macro F1-Score and Mean Absolute Error (MAE) of the predicted star rating, and provide a detailed analysis of the quantitative and qualitative differences.

Implementation Guidelines

Your implementation should include the following components:

1. Data Preprocessing

- **Baseline:** Implement a classic NLP preprocessing pipeline: lowercase and clean the text (HTML tags, URLs), tokenize (e.g., using spaCy or NLTK), build a vocabulary with a minimum frequency threshold, and load pre-trained fastText/GloVe vectors into an embedding matrix. Handle padding, unknown tokens and a fixed maximum sequence length (e.g., 200 tokens).

- **RoBERTa:** Use the `RobertaTokenizerFast` from the Hugging Face transformers library. This includes handling byte-level BPE tokenization, adding special tokens (<s>, </s>), truncation/padding to a maximum length (e.g., 256), and generating the `attention_mask`. Use dynamic padding with a data collator to speed up training.

2. Model Architecture

- **Baseline:** Embedding Layer (initialized with fastText/GloVe, frozen for the first epoch then unfrozen) -> Bidirectional GRU (2 layers) -> Additive Attention Pooling -> Dropout -> Linear Layer (output dims = 5).
- **RoBERTa:** Load a pre-trained `RobertaModel`. The classification model should compute the masked mean of the last hidden states, pass it through a Dropout layer (p = 0.1 to 0.3) and a final Linear Layer (output dims = 5).

3. Training Setup

- Train the models using Cross-Entropy Loss and an appropriate optimizer.
- **Baseline:** Use Adam with a standard learning rate (e.g., 1e-3), gradient clipping (max norm 1.0) and early stopping on validation Macro F1.
- **RoBERTa:** Use the AdamW optimizer with a low learning rate (e.g., 1e-5 to 3e-5), weight decay (e.g., 0.01), and a learning rate scheduler with 6-10% linear warmup followed by linear decay. Use mixed-precision (fp16/bf16) training and gradient accumulation if GPU memory is limited.
- Handle the large dataset efficiently using DataLoader (PyTorch) or tf.data (TensorFlow). Fix random seeds and report results as the mean and standard deviation over at least 2 seeds.

4. Ablation Study (choose any ONE)

- Mean pooling vs. first-token pooling for RoBERTa.
- Standard Cross-Entropy vs. class-weighted Cross-Entropy for either model.
- Maximum sequence length of 128 vs. 256 tokens for RoBERTa.

5. Evaluation

- Evaluate both models on the test set.
- **Quantitative:** Report overall Accuracy, Macro F1-Score, per-class F1-Score and MAE. Generate a normalized Confusion Matrix for both models and analyze class-specific errors, in particular how often each model confuses adjacent ratings (e.g., 4 vs. 5 stars) compared to distant ratings (e.g., 1 vs. 5 stars).
- **Qualitative:** Provide at least 5 examples from the test set where the baseline model fails but the RoBERTa model succeeds (and at least 2 where the opposite happens, if any). Analyze these examples (e.g., negation, sarcasm, mixed sentiment, long reviews, misspellings) to hypothesize why the models differ. For the transformer, visualize attention or token-level importance for at least one example.

Expected Deliverables

- Preprocessing and Data Loading notebook/script (must show handling for both classic embeddings and RoBERTa tokenization).
- Notebook for Baseline (BiGRU + Attention) Model: Training and Evaluation.
- Notebook for RoBERTa Model: Fine-tuning, Ablation and Evaluation.
- Graphs showing training/validation loss and Macro F1-score trends for both models.

- A concise report (maximum 6 pages) detailing the architecture, training process, ablation result, and a comparative analysis of the quantitative and qualitative results.

Evaluation Criteria (Total: 100 Marks)

#	Criterion	Marks
1	**Dataset Loading and Preprocessing:** Correctly loading the dataset, merging title and body, and implementing two distinct preprocessing pipelines (classic vs. RoBERTa).	10
2	**Baseline RNN Implementation:** A robust BiGRU model with attention pooling and pre-trained embeddings.	20
3	**RoBERTa Fine-Tuning Model:** Correctly loading the pre-trained model, implementing mean pooling and the classification head, and setting up the model for fine-tuning.	25
4	**Training Procedure and Optimization:** Efficient data handling, correct use of optimizers/schedulers (warmup, weight decay, mixed precision), and multi-seed reporting.	10
5	**Quantitative and Qualitative Evaluation:** Correct calculation and insightful comparison of Accuracy, F1-Scores, MAE and Confusion Matrices, supported by qualitative examples.	20
6	**Ablation Study:** A well-designed and clearly interpreted ablation experiment.	5
7	**Report Structure, Clarity, and Insight:** The quality of the final report in explaining the work and providing deep insights into the model comparison.	10
	Total	**100**

Question 3: Abstractive Text Summarization using the Transformer Architecture

Objective

The objective of this task is to develop an Abstractive Text Summarization model using a full Transformer Encoder-Decoder architecture. The goal is to process a news article in English and generate a short, coherent summary that captures its essential information. You will implement the key components of the Transformer model from scratch and evaluate its performance using standard summarization metrics.

Dataset

For this task, you will use the **CNN / Daily Mail** dataset, which contains over 300,000 unique news articles paired with multi-sentence highlights (summaries). This dataset is a standard benchmark for text summarization. Load version 3.0.0 from Hugging Face:

```
from datasets import load_dataset
ds = load_dataset("cnn_dailymail", "3.0.0")   # columns: article, highlights, id
```

Split	Article–summary pairs	Use it for
Training	287,113	training the model (and the tokenizer, if you train your own)
Validation	13,368	early stopping and choosing hyperparameters
Test	11,490	final ROUGE scores only — never for training or tuning

The `highlights` field is the reference summary: three to four sentences separated by newline characters (`\n`). Keep those newlines; they are needed for ROUGE-Lsum (see Evaluation).

Problem Statement

Implement a complete text summarization model based on the Transformer architecture using the CNN / Daily Mail dataset. You will build a **Transformer Encoder-Decoder** sequence-to-sequence model based on the original Transformer paper ("Attention Is All You Need").

- The **Encoder** reads the source news article and creates rich, contextual representations of it.
- The **Decoder** generates the summary token by token, attending to the encoder's output at each step to focus on the most relevant information.

You will evaluate the quality of the generated summaries on the test data using ROUGE scores and provide a qualitative analysis of the model's output.

> **What "from scratch" means.** You must write the attention, feed-forward, positional-encoding, encoder-layer and decoder-layer code yourself, using basic tensor operations and simple layers (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`, or their Keras/TensorFlow equivalents).
>
> **Not allowed:** `nn.Transformer`, `nn.TransformerEncoder(Layer)`, `nn.TransformerDecoder(Layer)`, `nn.MultiheadAttention`, `F.scaled_dot_product_attention`, `keras.layers.MultiHeadAttention`, any Hugging Face

> model class (BART, T5, Pegasus, …) and any pre-trained model weights. A pre-trained *tokenizer* is allowed.

Implementation Guidelines

Your implementation should include the following components:

1. Data Preprocessing: Tokenize and preprocess the text corpus using a sub-word tokenizer (e.g., SentencePiece, WordPiece, or a pre-trained tokenizer from Hugging Face). Your process should include lowercasing, creating a vocabulary, and padding sequences to a uniform length for batching.

Articles are long. Measured with the `bert-base-uncased` WordPiece tokenizer on 2,000 test articles:

	Median	90th percentile	Max	Note
Article	641	1,304	2,967	62.5% are longer than 512 tokens
Summary	49	74	213	only 0.5% are longer than 128 tokens

- **Truncate** the article to 400–512 tokens and the summary to about 128 tokens. News articles put the key facts first, so little is lost. State the limits you chose.
- If you lowercase, use an *uncased* tokenizer (or train your own SentencePiece model on lowercased training text). If you train a tokenizer, train it on the **training split only**.
- Use one shared vocabulary for articles and summaries. Add start and end tokens to every summary.
- Decoder input = summary shifted right (starts with the start token); prediction target = summary shifted left (ends with the end token).

2. Model Architecture: Implement the key building blocks of the Transformer from scratch:

- **Encoder:** A stack of identical layers, each containing a Multi-Head Self-Attention mechanism and a Position-wise Feed-Forward Network.
- **Decoder:** A stack of identical layers, each containing a Masked Multi-Head Self-Attention mechanism, an Encoder-Decoder Attention mechanism, and a Position-wise Feed-Forward Network.
- **Positional Encodings:** Implement and add positional encodings to provide the model with information about the order of tokens in the sequence.

Core formulas from the paper:

```
Attention(Q, K, V) = softmax( Q K^T / sqrt(d_k) + mask ) V
PE(pos, 2i)   = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i / d_model) )
```

- Every sub-layer is wrapped as `LayerNorm(x + Dropout(Sublayer(x)))`.
- Three masks are needed: a **source padding mask** (encoder self-attention and encoder-decoder attention), and a **causal (look-ahead) mask** combined with a **target padding mask** (decoder self-attention).
- A final linear layer maps the decoder output to vocabulary logits.
- A smaller model than the paper's base model is fine (for example d_model 256–512, 3–6 layers per stack, 4–8 heads). Report the configuration and parameter count.

3. Training Setup: Train the model using cross-entropy loss and the Adam optimizer, preferably with a learning rate scheduler. Use teacher forcing during training for stable and faster convergence. Implement early stopping based on the validation loss to prevent overfitting.

- Ignore padding positions in the loss (e.g. `ignore_index=pad_id`).
- The paper's schedule is warm-up then decay, with Adam β = (0.9, 0.98), ε = 1e-9:

```
lr = d_model^-0.5 * min(step^-0.5, step * warmup_steps^-1.5)
```

- Teacher forcing in a Transformer means feeding the gold shifted-right summary as the decoder input, so the whole summary is trained in one parallel pass. The causal mask stops each position from seeing later tokens.
- Early stopping: evaluate the validation loss after each epoch (or every N steps), stop after a fixed patience, and **restore the best checkpoint** before testing.
- If compute is limited, you may train on a subset of the training split. State the size you used.

4. Evaluation: Evaluate the model using the ROUGE metric on the test set. Specifically, you must report ROUGE-1, ROUGE-2, and ROUGE-L scores. Additionally, provide qualitative examples by comparing model-generated summaries with the ground-truth summaries.

- **Generation has no teacher forcing.** Start from the start token and decode autoregressively, feeding each predicted token back in, until the end token or a maximum length (e.g. 128). Greedy decoding is required; beam search is optional. Encode each article once and reuse the encoder output at every step.
- Convert token IDs back to text before scoring. Report **F1 scores on the full test set** (11,490 pairs). Use the `rouge_score` package (or `evaluate.load("rouge")`) with `use_stemmer=True`.
- **ROUGE-L vs ROUGE-Lsum.** Plain ROUGE-L treats the summary as one sequence; ROUGE-Lsum (the standard for CNN/DM) works sentence by sentence and needs sentences separated by `\n` in both the reference and your output. They give very different numbers, so report **both** and label them.

Reference point, measured on the full test set with `rouge_score` (stemming on). **Lead-3** takes the first three sentences of the article as the summary, with no model at all:

System	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
Lead-3 baseline	40.06	17.48	25.02	36.36

A Transformer trained from scratch with limited compute will usually score **below** Lead-3. This is normal. Compare against it and explain the gap in your report.

For the qualitative part, show at least five test articles with the reference and generated summaries. Include failure cases such as repeated phrases, wrong names or numbers, or facts not in the article.

5. Visualization: For at least one example, visualize the encoder-decoder attention weights as a heatmap. This should show which words in the source article the model focuses on when generating a specific word in the summary.

- Take the encoder-decoder (cross) attention of the **last decoder layer**, recorded while the model **generates** the summary (not during teacher-forced training). Average over heads or show heads separately; state which.
- Rows = generated summary tokens, columns = source tokens, with tokens as axis labels. A full 512-token article is unreadable, so show a window of the source (e.g. 50–100 tokens) where

most of the attention falls.

Expected Deliverables

You are required to submit a single compressed file (e.g., a .zip file) containing:

- **Source Code:** All Python scripts and/or Jupyter notebooks used for data preprocessing, model implementation, training, and evaluation. The code must be well-commented and organized logically. Include a README with run instructions and required libraries. Do not include the dataset or model checkpoints.
- **Final Report:** A comprehensive report that includes a detailed analysis of your work, results, graphs (e.g., training and validation loss curves), and the required attention visualizations. It must state the tokenizer, truncation lengths, model configuration, training-set size, decoding method, and ROUGE-1, ROUGE-2, ROUGE-L and ROUGE-Lsum on the test set.

Common Pitfalls

1. Using `nn.Transformer`, `nn.MultiheadAttention` or a pre-trained summarization model instead of writing the components.
2. A missing or wrong causal mask: training loss drops very fast, but generated summaries are nonsense.
3. Forgetting the padding mask, so the model attends to padding tokens.
4. Counting padding tokens in the loss.
5. Reporting ROUGE on teacher-forced outputs instead of on summaries the model generates by itself.
6. Reporting ROUGE-L without saying whether it is ROUGE-L or ROUGE-Lsum.
7. Tuning on the test set, or testing the last checkpoint instead of the best one.

Evaluation Criteria (Total: 100 Marks)

Marks will be distributed as follows:

#	Component	Marks
1	Dataset loading and preprocessing	10
2	Transformer Core Components (Multi-Head Attention, FFN, Positional Encoding)	20
3	Full Encoder-Decoder Model Implementation	20
4	Training procedure and optimization	10
5	Quantitative evaluation (ROUGE and loss curves)	15
6	Qualitative analysis and attention visualization	15
7	Report structure, clarity, and insight	10
	Total	**100**

Question 4: The Plot-Twist Generator

Can an AI Write an Interesting Story?

Objective

Can a small neural language model write a story that is coherent, interesting, and capable of surprising its reader?

In this assignment, students will develop a neural text-generation system that learns patterns from short stories and generates new story continuations from user-provided opening sentences.

The objective is not merely to generate grammatically plausible text. Students must investigate how model architecture and decoding strategies affect creativity, coherence, repetition, and the consistency of generated stories.

You will implement and compare Vanilla RNN and LSTM language models, test different text-generation strategies, and evaluate generated stories with quantitative metrics and human judgement. An optional open-source LLM may be added as an extra comparison baseline.

Dataset

Use the **TinyStories dataset**, available through the Hugging Face Datasets library.

```
from datasets import load_dataset

dataset = load_dataset("roneneldan/TinyStories")
```

For computational feasibility, use a documented subset of the available stories. A suggested configuration is 20,000 training stories, with separate validation and test sets of 1,000 stories each. You may use smaller subsets when hardware is limited, provided you document the sizes.

Alternatively, an instructor-approved collection of short stories may be used.

Split the data at the story level before creating training sequences. Do not let overlapping sequences from the same story appear in different splits.

Problem Statement

Build a system called the **Plot-Twist Generator** that takes the beginning of a story as input and generates a continuation.

For example:

Input prompt:

> “The detective entered the abandoned house and noticed that every clock had stopped at exactly midnight. Suddenly, the telephone rang…”

Possible generated continuation:

> “The detective picked up the phone, but nobody spoke. Instead, he heard the ticking of a clock. He turned around and saw a small clock lying on the floor. Inside it was a photograph of himself, taken only a few minutes ago.”

The generated continuation above is illustrative only. Your model must generate its own text.

Part 1 — Dataset Preparation and Next-Token Prediction

Prepare the story corpus for neural language modelling.

1. Clean and tokenise the text.
2. Construct a vocabulary and handle rare or unknown tokens.
3. Introduce appropriate beginning-of-sequence, end-of-sequence, padding, and unknown tokens as required by your tokenisation method.
4. Convert the stories into sequences of token IDs.
5. Construct input-target pairs so that the model predicts the next token from the preceding tokens.

You may use word-level or subword tokenisation. Both models must use the same tokenisation procedure and vocabulary to ensure a fair comparison.

Explain why story boundaries must be preserved and how padding is handled during training.

Part 2 — Build Two Neural Language Models

Implement the following models.

Model A: Vanilla RNN Language Model

Use the architecture:

```
Embedding → Vanilla RNN → Linear Layer → Vocabulary Logits
```

Model B: LSTM Language Model

Use the architecture:

```
Embedding → LSTM → Linear Layer → Vocabulary Logits
```

Train both models to predict the next token using cross-entropy loss.

Keep the dataset, vocabulary, embedding dimension, hidden dimension, and training conditions comparable wherever practical. Report any unavoidable differences.

Record the training and validation loss curves, model parameter counts, training duration, and final validation perplexity.

You may reuse foundational code from the neural language-modelling laboratory, but the story-generation experiments and comparative analysis must be new.

Part 3 — Make the Stories Come Alive

Implement at least three text-generation strategies.

1. **Greedy decoding:** Select the highest-probability next token at each step.
2. **Temperature sampling:** Adjust the probability distribution before sampling. Experiment with at least three temperatures, such as 0.5, 0.8, and 1.2.
3. **Top-k sampling:** Restrict the next-token choices to the k most probable tokens before sampling.

Generate story continuations using all three strategies from the same set of opening prompts.

Use at least five prompts, including examples involving a mystery, a surprising discovery, an unusual character, a problem that needs solving, and an unexpected ending.

For every prompt, generate multiple continuations under different decoding configurations.

Analyse the differences between the results. Investigate whether lower temperatures produce repetitive text, whether higher temperatures lead to incoherent continuations, and whether top-k sampling offers a useful compromise.

Part 4 — The Plot-Twist Challenge

Now test the system creatively.

Prepare at least five story openings designed to lead to a surprising development. Examples include:

- A student discovers that their examination paper contains answers to tomorrow's questions.
- An old robot receives a message from its future version.
- A detective solves a murder before the crime occurs.
- A village wakes up to discover that nobody can remember the previous day.
- A person receives a letter addressed to them from 100 years in the future.

For each opening, generate continuations using both trained models and at least two decoding strategies.

Evaluate whether each continuation:

1. Remains connected to the original prompt.
2. Introduces a meaningful development or twist.
3. Maintains consistency with details established earlier in the generated text.
4. Avoids excessive repetition and abrupt, meaningless transitions.

Do not manually edit the generated stories before evaluation. Clearly separate model-generated text from any human-written commentary.

Part 5 — Is the Better Language Model Also the Better Storyteller?

A lower perplexity does not necessarily imply that a generated story will be more interesting or coherent.

Conduct both quantitative and qualitative evaluations.

Quantitative evaluation

Report the following for the two models:

- Validation and test perplexity.
- Distinct-1 and Distinct-2, measuring the diversity of generated unigrams and bigrams.
- Repetition rate, using a clearly defined method.
- Average generated length and the frequency of prematurely terminated or excessively long outputs.

Evaluate the models using the same held-out test split and document the text-generation settings used to produce your samples.

Human evaluation

Select at least 10 generated stories or continuations from your experiments. Present them in a blinded evaluation where the evaluator is not told which model produced each sample.

Rate each sample on a five-point scale for:

- Coherence.
- Grammatical quality.
- Relevance to the opening prompt.

- Creativity and surprise.

Use at least three evaluators wherever feasible. Report average scores and discuss disagreements between evaluators.

Analyse whether the model with the lower perplexity also receives better human ratings. Explain possible reasons for any disagreement between automatic metrics and human judgement.

Part 6 — Final Analysis

Your report must answer the following questions using evidence from the experiments:

1. How do Vanilla RNN and LSTM differ in their ability to model sequential text?
2. Which model generates more coherent and less repetitive continuations?
3. How does temperature affect creativity and grammatical quality?
4. Can higher diversity scores guarantee better stories?
5. Does a lower perplexity correspond to a better storytelling ability in your experiments?
6. What kinds of failures occur most frequently, and how might they be reduced?
7. What are the limitations of generating stories with a small language model trained on a limited corpus?

Support your answers with examples, tables, and observations from your experiments.

Expected Deliverables

Submit a single compressed file containing:

- Source code and notebooks for preprocessing, training, evaluation, and generation.
- Trained model checkpoints and the vocabulary or tokenizer needed to reproduce the results.
- Training and validation loss curves.
- Quantitative comparison tables.
- Generated stories for the Plot-Twist Challenge.
- Human-evaluation results and qualitative error analysis.
- A technical report explaining the methodology, experimental results, and conclusions.
- A README containing setup instructions and commands required to reproduce the experiments.

Do not include the full dataset in the submitted archive. Document any resource limitations and the subset used.

Optional Extension - Open-Source LLM Comparison

Evaluate one small open-source pretrained LLM (for example, Qwen2.5-0.5B-Instruct, TinyLlama-1.1B-Chat, or SmolLM2-360M-Instruct) as an additional baseline. Use the same five or more story openings used in the Plot-Twist Challenge.

Use zero-shot or few-shot prompting; do not fine-tune the LLM on TinyStories. Record the exact model checkpoint, prompt template, decoding settings, precision or quantization, and hardware. Keep prompts and generation limits comparable across systems.

Present the LLM outputs beside the Vanilla RNN and LSTM outputs. Evaluate all outputs using the same criteria in Part 5 (coherence, grammatical quality, prompt relevance, creativity, and surprise), and report blinded human ratings where feasible. Compare common failure types and repetition/diversity metrics where applicable; perplexity may not be directly comparable across different tokenizers.

This extension is optional: the LLM does not replace either from-scratch model and does not change the 100-mark rubric. If compute is limited, choose a smaller checkpoint and document the limitation.

Evaluation Criteria — Total: 100 Marks

Component	Marks
Dataset preparation and next-token training examples	10
Correct Vanilla RNN and LSTM implementations	25
Training procedure and quantitative model evaluation	15
Implementation and analysis of decoding strategies	15
Plot-Twist Challenge and generated-story analysis	10
Diversity metrics and blinded human evaluation	15
Report quality, reproducibility, and technical conclusions	10
Total	**100**

Important Notes

- Do not use a pretrained LLM to replace the required Vanilla RNN and LSTM models. An open-source LLM is allowed only as an additional comparison baseline.
- Train both the Vanilla RNN and LSTM on the selected corpus. The optional LLM may be evaluated without fine-tuning.
- Clearly record the random seeds and generation parameters for reproducibility.
- Do not claim that a story is better solely because it has higher diversity or lower perplexity.
- Failed, repetitive, or incoherent generations must be included in the analysis rather than omitted.
- Marks will be awarded for well-designed experiments and evidence-based conclusions, not simply for producing the most entertaining story.